

HPC Lab3.5 Report

1 实验目标	2
2 算子语义与硬件背景	2
2.1 FusedAddRmsNorm	2
2.2 昇腾 910B 达芬奇架构要点	2
3 性能基线	2
4 优化迭代	3
4.1 消除 PIPE_ALL 串行化 (V2)	3
4.2 批量 chunk 路径 (V4)	3
4.3 标量 rstd (V5)	4
4.4 raw TBuf 替换队列簿记, weight 移出关键路径 (V9)	4
4.5 Tiling 与 chunk 宽向量化 (V13)	4
4.6 最终实现的数据流 (V28c)	4
5 第二轮冲刺: 评分曲线复核与地板分解 (V14-V30)	5
5.1 评分曲线的像素级重拟合	5
5.2 Task Duration 的地板分解	5
5.3 保留到最终版的修改 (V20-V28)	6
6 尝试过但未采用的方案	6
7 最终结果	7
7.1 正确性	7
7.2 性能	7
8 思考题	8
8.1 GM 读写量、FLOP 与算术强度	8
8.2 Double Buffer 流水	8
8.3 TQue 是否真实队列	8

1 实验目标

本实验在华为昇腾 910B4 NPU 上实现并优化融合算子 FusedAddRmsNorm。官方评测 (V13 提交) Task Duration = 6.02 us, 得分 **76/120**, 相对 baseline 加速 **2.49 倍**。在此基础上我们进行了第二轮冲刺 (V14-V30), 交付 V28c: 本地中位 6.06 us, 并用受控实验论证了满分在本环境被结构性封顶 (详见后文地板分解)。

2 算子语义与硬件背景

2.1 FusedAddRmsNorm

设 x 、 residual 为 $B \times H$ 的 FP16 张量, w 为 H 维 FP16 权重。对每一行 b , 算子计算

$$R_b = x_b + \text{residual}_b, \quad \text{rms}_b = \sqrt{\frac{1}{H} \sum_{i=0}^{H-1} R_{b,i}^2 + \varepsilon}, \quad (1)$$

$$y_b = \frac{R_b}{\text{rms}_b} \cdot w, \quad \text{residual_out}_b = R_b \quad (2)$$

其中 $\varepsilon = 10^{-6}$ 。算子同时输出 y 与 residual_out 两个 $B \times H$ 张量。正确性判据为逐元素 $|o_i - g_i| \leq 10^{-3}$ 或相对误差 $\leq 10^{-3}$ 。

2.2 昇腾 910B 达芬奇架构要点

910B 系列 (A2) 的 AI Core 采用 AIC/AIV 分离设计: AIC 负责矩阵类运算, AIV 负责向量与元素级运算。本算子不含矩阵乘, 全部工作在 AIV 上。每个 AIV 核关键资源如下:

资源	规格	用途
UB (Unified Buffer)	192 KiB / AIV	向量指令操作数唯一存放处
Vector 单元	VLEN 256 B	Cast/Add/Mul/Reduce 等逐元素指令
MTE2	-	GM $\rightarrow$ UB 数据搬入
MTE3	-	UB $\rightarrow$ GM 数据搬出
Scalar 单元	-	标量运算、地址计算、控制流

Table 1: 910B4 AIV 核关键资源

MTE2、V、MTE3 三条流水线相互独立, 可以并行; 同步通过 TQue 队列语义自动插入的事件或显式 SetFlag/WaitFlag 完成。Vector 指令必须操作 UB 上的数据, FP32 按 256 B/次处理 64 个元素, FP16 处理 128 个元素, 32 B 对齐是向量指令与搬运的基本粒度。

选择 **Ascend C** 开发路径。

3 性能基线

课程提供的 Ascend C 基线 (下称 V0) 按行处理: inQueX/inQueRes (FP16, 双缓冲)、outQueY/outQueResOut 四个 TQue, 以及 weightHalf/weightFp32/resoFp32/sq/scalar 等 TBuf。每行数据流为: DataCopyPad 搬入 x 与 $\text{residual} \rightarrow$ Cast 到 FP32 并 Add 得 R (同时 Cast FP16 写 residual_out) $\rightarrow$ 平方与 Block/WholeReduceSum 规约 $\rightarrow$ GetValue 取标量后 Duplicate + Sqrt + Div + Mul(weight) $\rightarrow$ Cast FP16 搬出。同步方面, CopyIn/CopyOut 周围有大量 PipeBarrier<PIPE_ALL>, 使 MTE2/V/MTE3 跨行完全串行。

基线正确性 5/5 通过。case 2 性能 5 次顺序测量：

15.1012, 14.7812, 15.0412, 15.2812, 15.1200 us

baseline 合计 8 个样本（含同条件复测 3 次）中位 **15.01 us**（范围 14.58-15.28）。baseline 的 msprof (aic-metrics=Default) 显示：Task 14.98 us, Block Dim 40, 核 0-35 各 7 行、核 36 仅 4 行、核 37-39 空转；流水占比中位 vec 19-23% (2.72 us)、scalar 37-51% (4.7-7.1 us)、MTE2 35-42% (4.2-5.5 us)、MTE3 14-16% (1.9 us)；L2 命中率 6.6%，GM 到 UB 带宽利用率约 2%。没有任何流水线被占满，说明瓶颈是串行化、同步与负载不均，而非计算或带宽。

4 优化迭代

版本	主要修改	5 case	case2 中位 / us	相对基线
V0 baseline	按行 TQue + PIPE_ALL	5/5	15.01	1.00×
V2	去除 CopyIn/Out 的 PIPE_ALL, TQue 双缓冲生效	5/5	9.92	1.52×
V4	批量 chunk 路径: chunk 级拷贝、R 留 UB、一次 V_S 同步	5/5	7.62	1.98×
V5	rstcd 全标量, 省每行 3 个微型向量操作	5/5	7.60	1.97×
V9	批量路径去 TQue, raw TBuf + 手动事件, weight 移出关键路径	5/5	6.94	2.16×
V13 final	tiling: blockDim=32 + rowsPerChunk=8; chunk 宽 phase A/B	5/5	6.28	2.39×

Table 2: 优化版本汇总（case 2, checker/profile.sh 中位数）

4.1 消除 PIPE_ALL 串行化（V2）

瓶颈：baseline 的 msprof 显示 MTE2/V/MTE3 均远未占满（利用率 19-51%），scalar 与同步开销反而最高，说明 PipeBarrier<PIPE_ALL> 把三条流水线完全串行化。

修改：去掉 CopyIn/CopyOut 周围的 PIPE_ALL 与冗余 V_MTE3 flag，只依赖 TQue EnQue/DeQue 自动插入的 MTE2→V 与 V→MTE3 同步事件，并配合编译器 --cce-auto-sync 处理 V 内部 RAW 依赖。

收益：中位 15.01 → 9.92 us (-34%)。TQue 的队列事件提供了正确的跨流水同步，而 PIPE_ALL 是多余的全局屏障。

4.2 批量 chunk 路径（V4）

瓶颈：simulator 显示单行 (H=4096) 约 852 条 SCALAR 指令，其中约 85% 是队列 API、地址与掩码 setup；逐行的队列调用是 scalar 主要开销，且逐行小拷贝无法利用 MTE2 带宽。

修改：对齐且 $H \leq 4096$ 时走批量路径：一个 chunk 的多行用一条多块 DataCopy 搬入（行在 GM 中连续，blockCount=n, blockLen=H/16），R 保留在 UB，逐行规约写入 sumSqArr[row*8] (32 B 对齐 lane)，整个 chunk 只做一次 V→S 同步；输出 residual_out、y 各用一条多块 DataCopy 写回。

收益：中位 9.92 → 7.62 us (-21%)。msprof 中 MTE3 从 1.7 us 降到 0.24 us（chunk 级大拷贝），scalar 从 4.2-4.7 us 降到 2.6 us。

4.3 标量 rstd (V5)

瓶颈: 原实现对每行做 Duplicate + Sqrt + Div 三个微型向量操作, rstd 的计算串在 V 链上。

修改: 用标量内建 sqrt: $rstd = 1/\sqrt{\text{sumSq} \cdot \text{invH} + \text{eps}}$, 每行一次标量运算, 随后 Muls(R, rstd) 与 Mul(R, weight) 两个向量操作即可, rstd 完全移出 V 链。

收益: 中位 7.62 $\rightarrow$ 7.04 us (-8%)。精度方面, 标量 1/sqrt 与 golden 的 $R / \sqrt{\dots}$ 在 FP32 下相差约 1-2 ULP, 最终误差仍由 FP16 舍入边界翻转主导。

4.4 raw TBuf 替换队列簿记, weight 移出关键路径 (V9)

瓶颈: V5 的批量路径仍用 TQue 管理输入输出, 每 chunk 的 Alloc/EnQue/DeQue/Free 与基线 weight 加载处的 PipeBarrier<PIPE_ALL> 仍在消耗标量周期; 且 2 KB 的 weight 拷贝被 PIPE_ALL 放到 kernel 启动的关键路径上。

修改: 批量路径改用 raw TBuf + 显式 SetFlag/WaitFlag (每类事件用高位硬编码 ID, 如 EVENT_ID5, 避免与框架事件池冲突)。weight 拷贝首条发出、紧邻 Set/Wait(MTE2_V) 并 Cast, 与 chunk 数据拷贝互不阻塞。

收益: 中位 7.60 $\rightarrow$ 6.94 us。同时修复了原代码中 sumSqBuf 只按 8 个 float 分配、而每行按 32 B 步长写 sumSqArr[row*8] 导致的 UB 越界隐患。

4.5 Tiling 与 chunk 宽向量化 (V13)

瓶颈: V9 在 blockDim=40 下, 256 行被切成 16 个核各 7 行、24 个核各 6 行, 7 行核成为拖尾 (profile 中 aiv 最大值明显高于中位); 且 phase A/B 仍逐行发指令 (每行 5 个元素级操作 + 3 个规约操作)。

修改 (host 与 kernel 协同):

1. blockDim 设为 $\min(\text{AIV}, B, 32)$ 。256 行 = 32 核 $\times$ 8 行, 负载完全均衡, 同时减少 kernel 分派开销 (课程文档的空 kernel 图显示分派成本随核数线性增长);
2. rowsPerChunk 预算从 144 KiB 提到 160 KiB, H=1024 时 chunk=8, 每核恰好 1 个 chunk, 避免跨 chunk 同步;
3. phase A 改为 chunk 宽: 行在 UB 中连续, Cast/Cast/Add/Cast/Mul 由每行 5 次变成每 chunk 5 次 (8 行合并); phase B 的 y 输出 Cast 也合并为 chunk 宽一次。

收益: 中位 6.94 $\rightarrow$ 6.28 us (官方评测 6.02 us, 76 分)。V13 的 msprof: Task 6.54 us, aiv 中位 5.00 us (最大 5.89), vec 1.52 us, scalar 2.33 us, MTE2 1.51 us, MTE3 约 0.26 us。

版本	Task / us	aiv / us	vec / us	scalar / us	MTE2 / us	MTE3 / us
V0 baseline	14.98	12-14	2.7	4.7-7.1	4.2-5.5	1.9
V5	7.42	5.22	1.56	2.62	1.81	0.24
V13 final	6.54	5.00	1.52	2.33	1.51	0.26

Table 3: 各版本 msprof 中位流水时间

4.6 最终实现的数据流 (V28c)

对齐批量路径 (H 为 16 的倍数且 $H \leq 4096$) 的最终数据流:

```

tiling 直读 (5 次标量 GM 读, 替代 GET_TILING_DATA 宏)
GM x/residual → (2 条多块 DataCopy, blockCount=n, blockLen=H/16) → inBuf(F16, x|res)
→ Add(F16) → residual_out (F16 chunk 缓冲; FP16 加法与 golden 位级一致)
→ Cast → R32 → Mul sq = R^2
→ per-row 跨步 Add 折叠到行首 64 lane → 一次批量 WholeReduceSum → sumSqArr[row]
→ 1 次 V→S 同步 → per-row 标量 rstd = 1/sqrt(sumSq*invH+eps)
→ per-row Muls(R, rstd); Mul(R, weight) → chunk 宽 Cast → y
→ residual_out、y 各 1 条多块 DataCopy 写回 GM

```

weight 拷贝与 chunk 输入共用一个 MTE2_V 事件（其 Cast 在 phase A 后发出，远早于 phase B 首次读取）；chunk 数据拷贝后 Set/Wait(MTE2_V)；规约后一次 V→S；输出前紧邻 Set/Wait(V_MTE3)；多 chunk 时 chunk 边界 Set/Wait(MTE3_V) 与 V_MTE2 保护单缓冲复用。非对齐 $H \leq 4096$ 的 shape 走 chunked 两遍流式路径， $H > 4096$ 同样。精度保持 y 全程 FP32 + 标量 rstd，最大相对误差由最终 FP16 舍入边界翻转决定，理论有界 $9.77e-4$ ，恒小于 $1e-3$ 。

5 第二轮冲刺：评分曲线复核与地板分解（V14-V30）

在 76 分的基础上，我们针对“满分还需要多快”做了两件事：重新标定评分曲线，并用受控实验把 Task Duration 分解为三部分。

5.1 评分曲线的像素级重拟合

对课程 score.png 的全部曲线点做最小二乘拟合，得

$$\text{score}(T) = 81 \ln\left(\frac{15.2}{T}\right), \quad (3)$$

其中 T 为 case 2 的 msprof Task Duration (us)，拟合 RMSE 为 0.91 分。验证： $T = 6.02$ us 对应 75 分，与官方 76 分在量化误差内一致。由曲线：100 分对应 4.42 us，**120 分对应 3.45 us**。

5.2 Task Duration 的地板分解

三个受控实验（相邻 job 交替测量，消除设备状态漂移）：

1. **分派地板**：纯 return 的空 kernel (.o 约 6 KiB) Task 稳定在 2.40-2.62 us，且在相隔 5 小时的两个时段复测一致；
2. **tiling 首读**：读一次 tiling 字段后立即 return（真实二进制，10-18 KiB 任意大小）为 3.30-3.52 us。即首次 GM 标量读约 0.9 us 的 HBM 延迟 完全暴露在关键路径上（后续字段读命中 L2，几乎免费）；
3. **二进制大小假设否定**：6.1 / 10.2 / 18.5 KiB 三种大小的内核地板 相同，代码体积不是地板的成因；
4. **计算 body**：完整 kernel 5.9-6.1 us，即 body 约 2.5 us。

因此 **Task = 分派 2.48 + tiling 首读 0.9 + body 约 2.5**。120 分阈值 3.45 us 在此地板上只剩约 0.07 us 计算预算，任何正确实现都无法达到；100 分（4.42 us）要求 body 压到 1.04 us 以下，而 V13 的 V 流水线有效计算时间就有 1.52 us（V22b 降为 1.19 us），仅指令发射开销已超出预算。本环境的现实上限 约为 85-90 分。

5.3 保留到最终版的修改 (V20-V28)

1. **fold + 批量规约 (V20)**: 仿 CANN LayerNorm 的 LayerNormReduceSumImpl, 每行用一条跨步 Add (dstRepStride=0 累加) 把平方和折叠到行首 64 lane, 再用一次 WholeReduceSum (repeatTime=行数, dstRepStride=1 元素) 打包产出全部行的 sumSq, 替换 V13 每行两条 BlockReduceSum + 一条 WholeReduceSum 加四次 mask API 的链路;
2. **直读 tiling (V21)**: 用 5 次标量 GM 读替代框架 GET_TILING_DATA 宏。宏内部走 MTE2 → UB → 两次跨流水事件 → 栈拷贝, 标定显示贵约 1.1 us;
3. **fp16 Add 产出 residual_out (V22b)**: 两个 FP16 之和在 FP32 中精确、再舍入回 FP16 与 FP16 加法语义位级一致, 故直接在 FP16 上做加法即得 residual_out (golden 同值), 再 Cast 上来供后续计算。vec 流水时间从 1.52 降到 1.19 us;
4. **weight 并入输入事件 (V28a)**: weight 拷贝与 chunk 输入共用一个 MTE2_V 事件, 省一对 Set/Wait;
5. **删除 whole-row 路径 (V28b/c)**: 非对齐 $H \leq 4096$ 的 shape 改走 chunked 两遍流式路径 (只影响非计分 shape 的性能, 不影响正确性), 精简结构。

6 尝试过但未采用的方案

1. 向量 Rsqrt (V7): 910B 向量 Rsqrt 精度约 0.2-0.5% (error_ratio 0.19-0.47), 5 个公开 case 中 4 个 FAIL, 因精度拒绝。
2. 更小的 chunk (V6, rowsPerChunk=3): 更多 chunk 的每-chunk 同步/API 开销大于 MTE2/V 重叠收益 (7.80 vs 7.04 us), 回退。
3. 软件流水 + y 预乘 weight (V8): 7.46-7.58 us, 比 V5 慢, 回退。
4. residual_out 的 MTE3 提前到 phase A 后 (E-C): 噪声级, 未采纳。
5. rstd 循环拆分 (V16, 先一次性 GetValue 全部 8 个标量再批量发向量指令): 6.86 us, 比 V13 慢, 疑似寄存器/栈副作用, 回退。
6. TQue 双缓冲管线 (V17): prologue 发首 tile、循环内 phase A 后立即发下一 tile 以重叠 MTE2 与 phase B, 5 case 全过但 6.50 us, 队列簿记开销大于重叠收益, 回退。
7. 多 tile 手动管线 (V10/V14/V15/V18/V19): 目标是把 MTE2 藏到 V 下面以逼近 5.08 us (90 分), 但多 tile 结构在 CANN 8.5.0 下反复出现 vector core timeout。诊断: 事件 flag 为“一次 Set 只能被一次 Wait 消费”的语义, 且 FetchEventID 与 TQue 内部事件共享从 0 开始的 ID 池, 手动事件与队列事件发生 FLAGID 冲突 (simulator 指令流显示内核末尾存在一个等不到 MTE2→V flag 0 的 V 等待)。由于无法静态验证编译器 auto-sync 与手动事件的全部交互, 最终放弃多 tile 管线, 采用稳定的单 chunk 结构。
8. 纯 FP16 y 路径 (V22): y 的三个 FP16 舍入叠加后最大相对误差约 $2.4e-3$, 超出 $1e-3$ 容限, 4 个公开 case FAIL, 精度拒绝。
9. BlockReduceSum 树 (V26): 设想用两条 BlockReduceSum 加一条多 repeat WholeReduceSum 共三条指令完成整 chunk 规约。单行 chunk 正确, 但多行打包 (srcRepStride 不足 8 block) 时硬件读错: B=64 时奇数行的 sumSq 恰好多出约 1/17 个 partial (simulator 复现)。结论: 多 repeat WholeReduceSum 要求 srcRepStride 至少 8 block (一行满 repeat), 折叠到 16 lane 再打包读取不合法。

10. wTile 广播 (V27): 用 UB→UB DataCopy (srcStride=0) 把 weight 复制成 n 行, 使逐行权重乘法合并为一条 chunk 宽指令。但 srcStride=0 的语义是“块间无间隔”即连续读取而非重读, 目的缓冲越界, H=1024 碰巧相邻内存合法而通过, H=512 时读到垃圾 (行 0 正确、行 1-7 全错), 拒绝。
11. 投机预取流水 (V30): 在读取 tiling 之前以猜测布局 (每核第 8k 行起 8 行、H=1024) 先把 x/residual/weight 投机搬入 UB, 便 tiling 首读的 0.9 us 延迟与 MTE2 流水重叠; 猜测命中则免拷贝, 未命中则重拷。计分 shape 快路径生效, 但对小张量的越界读依赖相邻虚拟内存恰好映射, 出现与布局相关的随机损坏与偶发 ERR99999 异常 (同 shape 单独跑通过、顺序跑失败), 越界读不安全, 拒绝。并目发现 MTE2 读侧无 MMU 边界检查不等于读越界无害。
12. 两半软件流水 (V24): 把 chunk 拆两半先发全部加载再计算, 5 case 全过但无收益 (约 6.0 us): 两段粒度太粗, 填充与排空吃掉重叠收益。
13. blockDim=40 (V28c 下复测): 中位 6.40 vs 32 核的 5.98 us, 负载不均 (256 行/40 核) 且分派更贵, 维持 32。

7 最终结果

7.1 正确性

最终版本通过全部 5 个公开 case (两次独立运行), 并独立于 checker 另测 22 个隐藏 shape 防护 case (含 7×5003 一类非对齐大 H、 $H > 4096$ 两遍流式路径、 $B = 1$ 单核、极小 $H=3/8$ 、 $[-1000, 1000]$ 大范围数据), 全部 PASS。官方评测 (5 公开 + 5 隐藏 case) 正确性通过。residual_out 在所有测试中均为 0 误差, y 最大相对误差 $9.7e-4$, 低于 $1e-3$ 容限。

7.2 性能

最终版本 V28c (第二轮冲刺后的交付版) case 2 的最新 5 个样本: 5.84, 6.32, 6.06, 6.08, 6.04 us, 中位 **6.06 us**, 单次 profile 5.96 us。官方 OJ 评测 (V13 提交) Task Duration = 6.02 us, 得分 **76/120**; V28c 相对 V13 本地中位低约 0.2 us, 按评分曲线换算约 78-80 分。

结合地板分解, 满分在本环境不可达: 分派 2.48 us 加 tiling 首读 0.9 us 共 3.38 us 的硬地板已仅比 120 分阈值低 0.07 us, 而规约、rstd 与逐行缩放的串行计算链是算法必需。相对 baseline 的加速比为 $15.01/6.06 = \mathbf{2.48 \text{ 倍}}$ (官方口径 6.02 us 对应 2.49 倍)。

指标	基线 V0	V13	最终 V28c	说明
正确性	5/5	5/5 + 22 隐藏 PASS	5/5 + 22 隐藏 PASS	本地验证
case2 中位 / us	15.01	6.28	6.06	本机 checker/ profile.sh
vec 流水 / us	2.7	1.52	1.19	msprof PipeUtilization
官方 Task / us	约 15	6.02	待重新提交	OJ 评测 513c599c-r7
官方得分	0	76/120	待重新提交	课程评分曲线

Table 4: 最终结果汇总

8 思考题

8.1 GM 读写量、FLOP 与算术强度

[256,1024]、FP16 下：读 x、residual 各 512 KiB，写 y、residual_out 各 512 KiB，合计 2 MiB GM 流量。每元素主要运算：残差加 1 次 Add、平方 1 次 Mul、规约约 1 次 Add、rstd 缩放 1 次 Muls、权重 1 次 Mul，约 5 FLOP，总计约 1.31 MFLOP。算术强度约 0.63 FLOP/B，是典型的访存类算子。

但 msprof 显示 GM 到 UB 带宽利用率仅约 2% (baseline) 到 20% 量级 (最终版)，Task 6.02 us 对应等效带宽约 2 MiB/6.02 us $\approx$ 350 GB/s，远低于 910B4 的 HBM 带宽。因此本实现既不是计算瓶颈也不是 HBM 带宽瓶颈，而是指令发射与同步延迟受限，MTE2 与 V 的串行链是主要耗时。

8.2 Double Buffer 流水

最终版 (V13) 在评测 shape 下每核 8 行、恰好 1 个 chunk，不存在跨 chunk 的双缓冲；其加速来自消除逐行队列 API 与同步开销。早期版本 (V4/V5) 使用 TQue 的 BUFFER_NUM=2 在 chunk 粒度做双缓冲，并用 msprof op simulator --soc-version=Ascend910B4 验证：560×1024 (每核 14 行、2 chunks) 的 core0 指令流显示 MOV_OUT_T0_UB (chunk2 输入，1047 cyc) 与 MOV_UB_T0_OUT (chunk1 输出，1047+685 cyc) 并发执行，即 chunk 级 MTE2/MTE3 确已重叠。

8.3 TQue 是否真实队列

TQue 不是硬件队列，而是编译期/运行期的“所有权与同步簿记”抽象。EnQue 把 LocalTensor 在 UB 中的地址与一个事件 ID 登记进 TPipe 的队列记录，并在对应流水线上发出 SetFlag；DeQue 取出同一地址的句柄并发出对应的 WaitFlag。数据本身始终留在 UB 中，EnQue/DeQue 之间不发生任何拷贝或搬移。